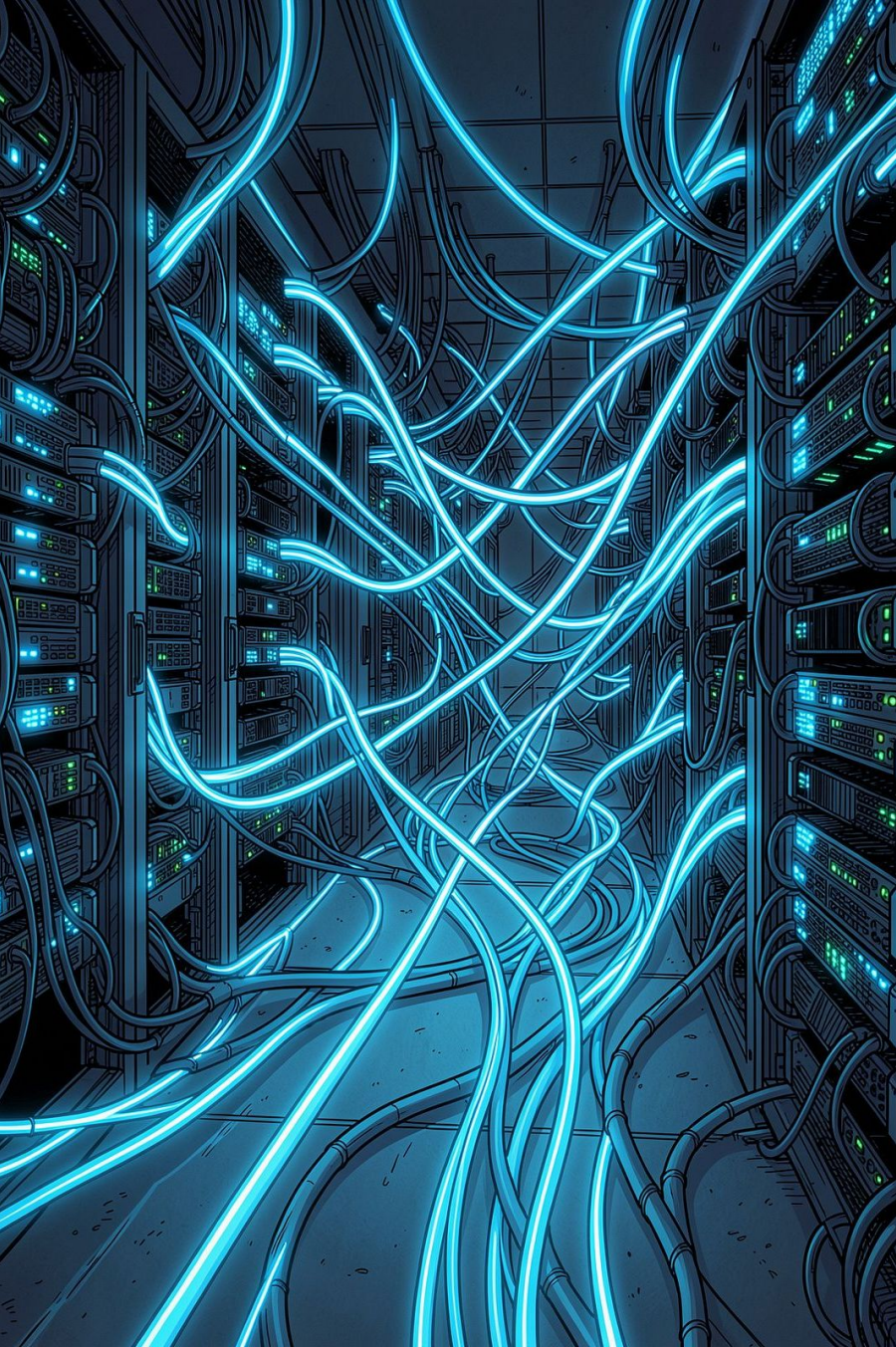




Aula 02: Arquitetura da Internet e Fundamentos de Back-end

Unidade I — Fundamentos Essenciais. Nesta aula, consolidamos a base lógica que sustenta todo sistema back-end moderno: da lógica imperativa clássica ao processamento de dados JSON, passando pelas metodologias que guiam equipes de desenvolvimento de alta performance.

UNIDADE I

AULA 02

+DESAFIO TÉCNICO 01

PROF. CLOVES ROCHA

0.1 Revisão: O Alicerce da Lógica Imperativa

Antes de construir sistemas complexos, é fundamental dominar os blocos elementares que compõem qualquer linguagem de programação. A **lógica imperativa** é o paradigma que orienta o computador a executar instruções passo a passo, descrevendo *como* algo deve ser feito — e não apenas *o que* deve ser feito.

Variáveis: Armazenando o Estado do Sistema

Uma variável é uma região nomeada na memória que armazena um valor que pode ser alterado durante a execução do programa. Em JavaScript, utilizamos `let` e `const` para declará-las. O tipo dinâmico permite que uma mesma variável armazene números, strings, objetos ou arrays sem declaração explícita de tipo. Compreender o escopo (global, de função ou de bloco) é essencial para evitar efeitos colaterais indesejados.

Estruturas Condicionais: O Poder da Tomada de Decisão

As estruturas `if/else` e `switch/case` permitem que o programa avalie condições booleanas e execute blocos de código distintos conforme o resultado. No back-end, condicionais são usadas para validar entradas, autorizar acessos, tratar erros e direcionar fluxos de negócio. O operador ternário (condição ? verdadeiro : falso) oferece uma sintaxe concisa para decisões simples.

Estruturas de Repetição: Automação e Eficiência

Os laços `for`, `while` e `do...while` automatizam a execução repetida de blocos de código. No contexto de back-end, são amplamente utilizados para iterar sobre coleções de dados, processar registros de banco de dados e implementar lógicas de retry para chamadas de API. Os métodos de array como `forEach`, `map`, `filter` e `reduce` oferecem alternativas funcionais e mais expressivas.

0.2 Manipulação de Dados: Funções e Conjuntos

O processamento eficiente de dados é a razão de existir de um back-end. Funções bem projetadas e o domínio de operações sobre coleções são habilidades que separam desenvolvedores júnior de profissionais plenos e sênior.

Funções: Modularização e Reutilização

Uma função é um bloco de código nomeado que executa uma tarefa específica e pode ser invocado múltiplas vezes. No JavaScript moderno, trabalhamos com **function declarations**, **function expressions** e **arrow functions**. A escolha correta impacta diretamente o comportamento do `this` e a legibilidade do código.

Boas práticas incluem: nomes descritivos, responsabilidade única (SRP), parâmetros com valores padrão e retorno explícito. Funções puras — que sempre retornam o mesmo resultado para os mesmos inputs e não geram efeitos colaterais — são a base de testes confiáveis.

Conjuntos e Mapeamento de Dados

Coleções como **Arrays**, **Set** e **Map** são estruturas fundamentais para organizar dados em memória. O `Array` é a mais comum, suportando métodos como `map()` (transformação), `filter()` (seleção), `find()` (busca), `reduce()` (agregação) e `sort()` (ordenação).

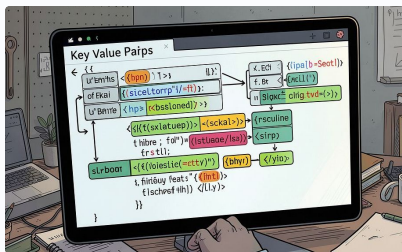
O `Set` garante unicidade de elementos — ideal para remover duplicatas. O `Map` permite chaves de qualquer tipo, sendo útil para caches e tabelas de lookup. Dominar esses métodos é pré-requisito para manipular respostas de APIs e dados de banco de dados.

O Padrão JSON

JavaScript Object Notation é o formato de intercâmbio de dados mais utilizado na web. Sua estrutura leve, legível por humanos e máquinas, e independente de linguagem, o tornou o padrão *de facto* para APIs REST, configurações e armazenamento de documentos.

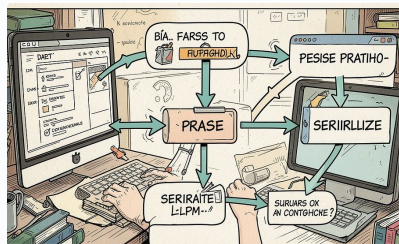
Processando o Universo JSON

O JSON é a língua franca da comunicação entre front-end e back-end. Compreender sua estrutura e saber manipulá-lo programaticamente é uma competência central para qualquer desenvolvedor back-end.



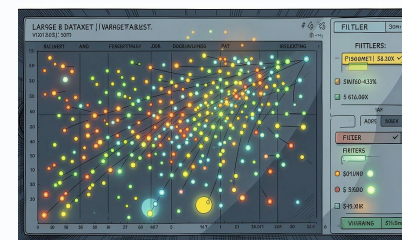
Estrutura Chave-Valor

O JSON organiza dados em pares de **chave** (string) e **valor** (string, número, booleano, array, objeto aninhado ou null). Essa simplicidade permite representar desde um usuário até uma árvore complexa de dados. Objetos aninhados e arrays dentro de arrays permitem modelar relações hierárquicas com fidelidade.



Parse e Serialização

Parse é o processo de converter uma string JSON em um objeto JavaScript utilizável (`JSON.parse()`). **Serialização** é o inverso: transformar um objeto em string para transmissão (`JSON.stringify()`). No back-end, toda requisição recebida precisa ser parseada e toda resposta precisa ser serializada antes de ser enviada ao cliente.



Filtragem Assíncrona de Grandes Volumes

Em cenários reais, o back-end processa milhares de registros. Combinar métodos de array (`filter`, `map`, `reduce`) com operações assíncronas (`async/await`, `Promises`) permite processar dados de forma não-bloqueante. Técnicas como paginação, lazy loading e streaming de JSON são essenciais para performance em produção.

- ❏ **Dica prática:** Sempre valide o JSON recebido com bibliotecas como `zod` ou `joi` antes de processá-lo. Dados não validados são a principal causa de bugs e vulnerabilidades em APIs.

0.3 Metodologias Ágeis: Scrum e Kanban

Desenvolver software com qualidade e previsibilidade exige mais do que habilidade técnica — exige **processo**. As metodologias ágeis surgiram como resposta à rigidez dos modelos tradicionais, priorizando entrega contínua de valor, adaptação a mudanças e colaboração intensa entre equipes.

Scrum: Estrutura e Ritmo

O Scrum organiza o trabalho em ciclos fixos chamados **Sprints** (geralmente 1 a 4 semanas). Cada sprint começa com o **Sprint Planning**, onde a equipe seleciona itens do **Product Backlog** para compor o **Sprint Backlog**. As **User Stories** descrevem funcionalidades do ponto de vista do usuário: *"Como [usuário], quero [ação] para [benefício]"*.

A **Daily Stand-up** é uma reunião diária de 15 minutos onde cada membro responde: O que fiz ontem? O que farei hoje? Há algum impedimento? Ao final da sprint, a **Sprint Review** demonstra o que foi entregue e a **Retrospectiva** identifica melhorias no processo.

Sprint

Backlog

Daily

Review

Kanban: Visualização e Fluxo

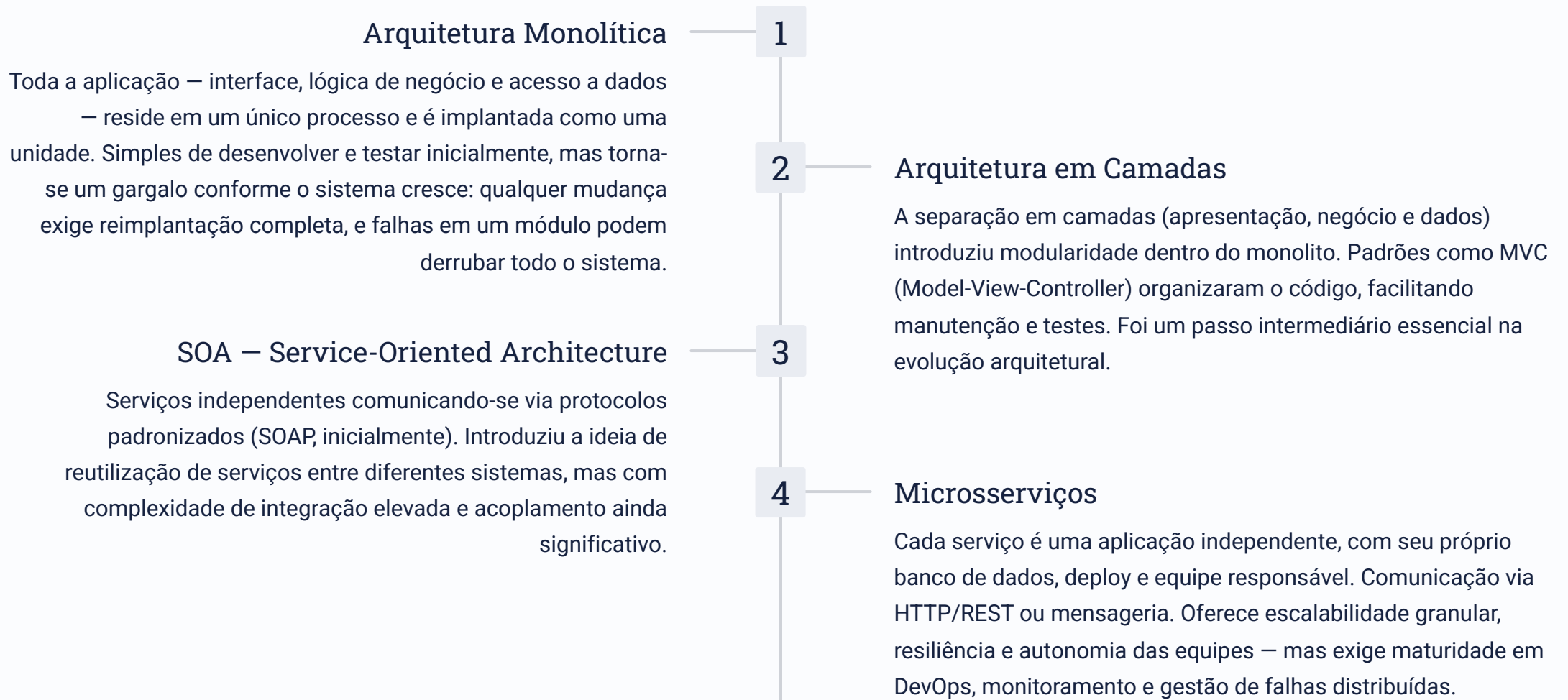
O Kanban é baseado na visualização do trabalho em um quadro com colunas que representam o estado das tarefas: **A Fazer** → **Em Progresso** → **Em Revisão** → **Concluído**. Cada tarefa é representada por um *card* que se move entre as colunas conforme avança.

O princípio central é o **WIP Limit** (Work In Progress): limitar o número de tarefas simultâneas em cada coluna para evitar sobrecarga e identificar gargalos rapidamente. Diferente do Scrum, o Kanban não possui sprints fixas — o fluxo é contínuo e puxado pela demanda.

A transição do planejamento estático (cascata) para o desenvolvimento adaptativo (ágil) significa aceitar que requisitos mudam e que entregar valor incrementalmente é mais seguro do que planejar tudo antes de começar.

Arquitetura da Internet: Do Monolito aos Microserviços

A forma como organizamos sistemas de software evoluiu drasticamente nas últimas décadas. Entender essa evolução é fundamental para tomar decisões arquiteturais conscientes e evitar armadilhas comuns em projetos reais.



O **protocolo HTTP** é o alicerce de toda essa comunicação. Stateless por natureza, baseado em requisição-resposta, define métodos (verbos), cabeçalhos e códigos de status que padronizam a interação entre clientes e servidores na web moderna.

O Estilo Arquitetural REST

REST (Representational State Transfer) é um estilo arquitetural definido por Roy Fielding em 2000 que estabelece restrições para criar APIs escaláveis, simples e interoperáveis. Não é um protocolo, mas um conjunto de princípios que, quando seguidos, resultam em sistemas mais previsíveis e fáceis de consumir.



Stateless: O Servidor Não Guarda Estado

Cada requisição deve conter **todas as informações** necessárias para ser processada. O servidor não armazena estado da sessão do cliente entre requisições. Isso simplifica a escalabilidade horizontal (qualquer servidor pode atender qualquer requisição) e aumenta a visibilidade das interações. A autenticação, por exemplo, deve ser enviada em cada request — geralmente via token JWT no header `Authorization`.



Interface Uniforme: Verbos HTTP Corretos

REST aproveita os verbos HTTP para expressar a intenção da operação: `GET` para leitura (seguro e idempotente), `POST` para criação, `PUT` para substituição completa, `PATCH` para atualização parcial e `DELETE` para remoção. Usar o verbo correto torna a API previsível e permite que proxies e caches otimizem o tráfego.



Recursos e URIs: A Identidade da API

Em REST, tudo é um **recurso** identificado por uma **URI** (Uniform Resource Identifier). URIs devem usar substantivos no plural (`/users`, não `/getUsers`), hierarquia para relações (`/users/123/orders`) e evitar verbos. A lógica da operação está no verbo HTTP, não no caminho. Códigos de status como 200, 201, 400, 401, 404 e 500 comunicam o resultado de forma padronizada.

Princípios complementares: REST também prega **cacheabilidade** (respostas devem indicar se podem ser cacheadas), **interface em camadas** (o cliente não sabe se está conectado diretamente ao servidor final) e **code on demand** (servidores podem enviar código executável, opcional).

Desafio Técnico 01: O Início da Jornada

Este é o momento de transformar conhecimento teórico em código real. O Desafio Técnico 01 foi desenhado para integrar todos os conceitos desta aula em um cenário prático e colaborativo — exatamente como funciona o desenvolvimento profissional.



Formação das Equipes

Cada equipe é composta por **6 estudantes** com um objetivo comum. A diversidade de habilidades dentro do grupo é uma vantagem: distribua papéis como líder técnico, responsável pela documentação, revisor de código e gestor do quadro Kanban. Times multidisciplinares entregam com mais qualidade.

6

Estudantes por Equipe

Equipes multidisciplinares com papéis definidos



A Missão Técnica

Aplicar **lógica de repetição e manipulação de JSON** em um cenário real de back-end. O desafio simula uma API que recebe, processa e transforma grandes volumes de dados — exigindo uso de loops, métodos de array, funções modulares e tratamento de erros. A solução deve ser testável e bem estruturada.

3

Tópicos Integrados

Lógica, JSON e Metodologias Ágeis em um único desafio



Integração com Metodologias Ágeis

O vocabulário aprendido — **Sprint, Backlog, User Stories, Daily e Kanban** — não é apenas teórico. As equipes devem organizar o trabalho usando um quadro Kanban (físico ou digital como Trello/Jira), realizar dailies e entregar incrementos ao final de cada ciclo. A prática ágil começa agora.

1

Objetivo Comum

Entrega colaborativa com qualidade e rastreabilidade

Dinâmica em Equipe: A Regra do Sucesso

O sucesso em projetos de software raramente é resultado de talento individual — é consequência de **processos bem executados**, comunicação clara e disciplina coletiva. Esta seção define as regras do jogo para o Desafio Técnico 01.

Kanban: O Quadro que Guia o Trabalho

Cada equipe deve criar e manter atualizado um **quadro Kanban** com as seguintes colunas mínimas:

- **Backlog:** Todas as tarefas identificadas, priorizadas pelo Product Owner da equipe
- **A Fazer:** Tarefas selecionadas para o ciclo atual (respeitar WIP Limit)
- **Em Desenvolvimento:** Código sendo escrito ativamente
- **Em Revisão (Code Review):** Código pronto para revisão de pares
- **Concluído:** Funcionalidade testada e integrada

O quadro deve ser atualizado *antes* de cada Daily. Um card parado por mais de um dia é um sinal de alerta que deve ser discutido imediatamente.

Versionamento: Garantia de Entrega

O uso do **Git** é obrigatório. Cada equipe deve criar um repositório com estrutura clara de branches: `main` (estável), `develop` (integração) e *feature branches* (feature/nome-da-tarefa). Pull Requests com revisão de pelo menos um colega são obrigatórios antes do merge. Commits devem seguir o padrão Conventional Commits (feat:, fix:, docs:).

Arquitetura Primeiro, Código Depois

Antes de escrever uma linha de código, a equipe deve definir:

- Quais funções existirão e qual a responsabilidade de cada uma
- Qual a estrutura do JSON de entrada e de saída
- Como os dados fluirão entre as funções (pipeline de processamento)
- Quais casos de erro precisam ser tratados

Código sem arquitetura é dívida técnica desde o primeiro dia.

Conclusão e Próximos Passos

Ao longo desta aula, construímos uma base sólida que conecta lógica de programação, manipulação de dados e processos ágeis — os três pilares que sustentam o desenvolvimento back-end profissional. O caminho agora é claro: **prática deliberada**.

1 O Back-end como Motor da Experiência

Toda interface bonita, toda animação fluida, toda resposta instantânea depende de um back-end bem arquitetado. É no servidor que os dados são validados, transformados, persistidos e entregues com segurança. Desenvolvedores back-end são os engenheiros que constroem a fundação sobre a qual a experiência do usuário é erguida — trabalho invisível, mas absolutamente crítico.

2 Seu Código é a Fundação de Sistemas Robustos

O Desafio Técnico 01 não é um exercício acadêmico isolado — é um microcosmo do que vocês encontrarão no mercado. Sistemas reais são construídos por equipes, com processos, sob pressão de prazo e com requisitos que mudam. A qualidade do código que vocês produzem hoje — legível, testável, modular — é o que diferencia sistemas que escalam de sistemas que colapsam.

3 Comecem com Foco em Coesão e Baixo Acoplamento

Dois princípios arquiteturais guiarão toda a Unidade I: **Alta Coesão** (cada função faz uma coisa e faz bem) e **Baixo Acoplamento** (módulos dependem o mínimo possível uns dos outros). Aplicados desde o início, esses princípios tornam o código mais fácil de testar, manter e evoluir. Iniciem o desafio com essas lentes — o resultado será visível na qualidade da entrega.

✓ Próxima Aula:

1.1 Estruturação Analítica e Protocolos de Transmissão da Rede Global

1.1.1 O modelo arquitetural Cliente-Servidor e os ciclos de Requisição e Resposta

1.1.2 Dissecção estrutural do Protocolo HTTP/HTTPS, cabeçalhos e códigos de status

1.1.3 Transição de arquiteturas monolíticas para microsserviços de dados

- Aprofundaremos em Flask Python — o ambiente de execução e o miniframework que darão vida prática a tudo que aprendemos hoje. Tragam suas dúvidas do Desafio 01 para a próxima aula!